EECS281 Final

Tree

Def * root with no parent * unique parent
* can be empty

Terms root, parent, child, sibling
internal/external node (leaf)

depth(empty) = 0
depth(root) = 1
depth(v) = depth(p) + 1

height(empty) = 0
height(leaf) = 1
height(v) = max(height(child)) + 1

ordered tree: 兄弟有大小

binary tree
- proper~: all nodes have 0/2 child
- complete: every level filled except the last; to the left!!

Implementation

* array root: idx = 1 left c: idx = 2i right c: idx = 2i+1
  some indices can be skipped
* pointer based.

general tree → binary tree
left: new generation right: sibling

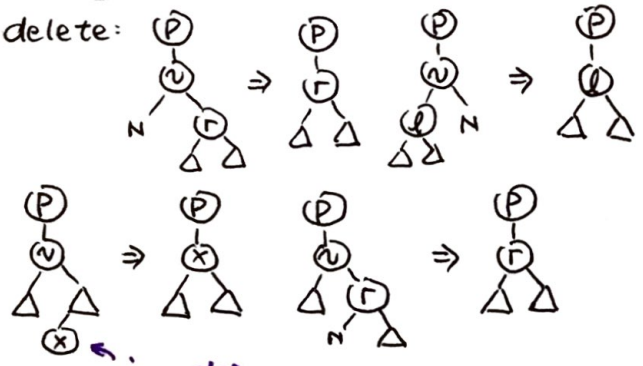
Tree traversal **DFS** use queue

preorder	inorder	postorder
visit(v)	inorder(l)	postorder(l)
preorder(l)	visit(v)	postorder(r)
preorder(r)	inorder(r)	visit(v)

BFS level order use stack

Binary Search Tree 左边 < 自己 < 右边

delete:



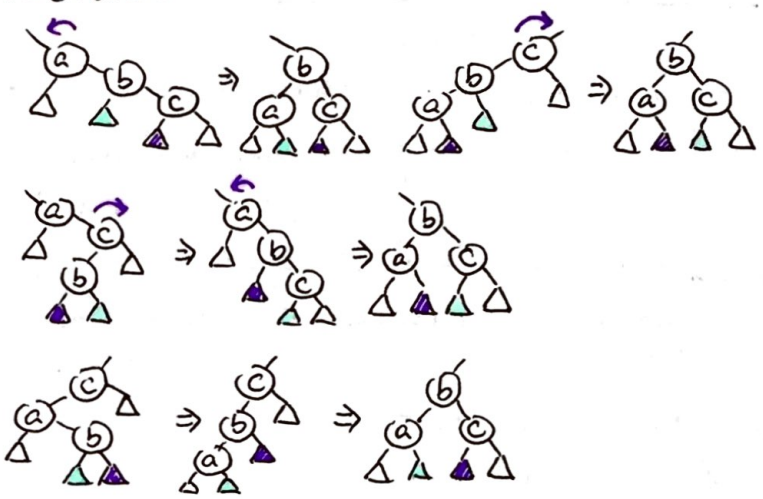
search O(h)
sort O(n)
insert O(h)
delete O(h)

AVL Tree

height = O(log(n))

balance(v) = height(l) − height(r) ∈ [−1, 1]

rotation:



Hash

* efficient for insert, search, remove
* inefficient for sort, select

- hash function h(key) = addr
- hash code t(key) = intmap
- compression map c(intmap) = addr

Collision Resolution

Seperate Chaining: insert O(1), search/remove O(N/M)
load factor α can be > 1.

Probing: Empty, Deleted, Hit, Full
- Linear~
- Quadratic~: (h(key) + j²)
- Double Hashing~: h(key) + j * h'(key)
  h'(key) = q − (h(key) % q)

Graph

vertex, edge

simple~: no parallel edge; no self-loop

- sparse~: |E| ≈ |V| adjacency list O(V+E)
- dense~: |E| ≈ |V²| adjacency matrix O(V²)

- directed
- undirected

- weighed
- unweighed

- DFS (O(V+E) list, O(V²) matrix)
- BFS

finds shortest path

MST

* 1st, 2nd shortest edge in MST

Prim

kv: isVisited
dv: min-dist
pv: parent

① select min dv
② kv = true
③ update dv, pv for all v.

- linear for dense O(V²)
- pq for sparse O(E log V)

Kruskal relies on efficiency of sort, union, find

sort edges. $O(E\log E) = O(E\log V)$

Union Find Data Structure

* representative!!!
* no need to store actual representative

union() be faster, find() be slower ⇒ yet union() still slower $O(N\alpha(N))$

TSP

Shortest Hamilton cycle (NP hard)

lower bound (MST) + 2 additional edge < TSP < higher bound (FAST TSP)

```
genPerm(deque u, vector p){
    if(u.empty()){ //^^}
    for(int k=0; k!=u.size(); k++){
        p.push_back(u.front());
        u.pop_front();
        genPerm(u, p);
        u.push_back(p.back());
        p.pop_back();
    }
}
```

Shortest Path

Dijkstra

kv. dv. pv. similar to MST except dv.

complexity: $O(V^2)$ or $O(E\log V)$

Algorithm

- Brute Force
- Greedy
- Divide & Conquer: ⇒ non-overlapping ~
- Dynamic ~: ⇒ overlapping subproblems $O(2^n) \to O(n^c)$
- Backtracking: for constraint satisfaction
- Branch & Bound: for optimization

e.g. Counting Change
- Brute Force $O(n2^n)$
- Greedy $O(n)$ if sorted

e.g. N-Queen

e.g. M-coloring

e.g. Knight Moves

e.g. Knapsack

e.g. binomial $\binom{n}{m} = \binom{n-1}{m-1} + \binom{n-1}{m}$

$O(nm)$ $\binom{n}{0} = \binom{n}{n} = 1$

Master Theorem

$$\begin{cases} T(n) = aT(n/b) + f(n) \\ T(n) = 1,\ n=1 \end{cases} \Rightarrow T(n) = \begin{cases} O(n^{\log_b a}),\ a > b^c \\ O(n^c \log_2 n),\ a = b^c \\ O(n^c),\ a < b^c \end{cases}$$

最高次为c

Sorting

	time	mem	stable?
bubble	n/n^2	1	✓
selection	n^2	1	×
insertion	n/n^2	1	✓
merge	$n\log n$	n	×
quick	$n\log n$	$\log n$	×
heap	$n\log n$	1	×